

 TECHNICAL DOCUMENTATION · V1.0

FixFLOW AI

RAG-Based Financial Operations Dashboard — architecture reference, data flow, security model, and feature walkthrough.

LIVE APP

fixflow-ai-z7zj.onrender.com ↗

STACK

React + Vite · Node/Express · MongoDB

AI ENGINE

Google Gemini API (5-model rotation)

TYPE

FinOps · Observability · RAG Pipeline

TABLE OF CONTENTS

- | | |
|---|---|
| 01 Executive Overview | 02 System Architecture |
| 03 Data Flow & Pipeline | 04 Security & PII Masking |
| 05 Feature Walkthrough | 06 RAG Knowledge Base |
| 07 Setup & Deployment | 08 Module Reference |

Executive Overview

FixFLOW AI is an intelligent Financial Operations (FinOps) dashboard purpose-built for banking data pipeline reliability. Modern financial systems generate complex, jargon-heavy error logs that non-technical operations teams cannot interpret — and sending raw payloads to cloud AI platforms creates serious compliance exposure (credit card numbers, emails, account IDs).

FixFLOW AI closes both gaps: it translates cryptic codes like `DEAD_LETTER` into plain-language alerts, masks all PII before any data leaves the cluster, and augments responses with locally-indexed Standard Operating Procedures via a TF-IDF/cosine similarity RAG pipeline.



Jargon Translation

Converts raw stack traces & error codes into friendly, actionable operations alerts.



Recursive PII Masking

Scrubs credit cards, emails, accounts, and phone numbers from all prompts before external dispatch.



Local RAG Retrieval

Indexes SOPs locally with cosine similarity search to augment AI context without external data leaks.

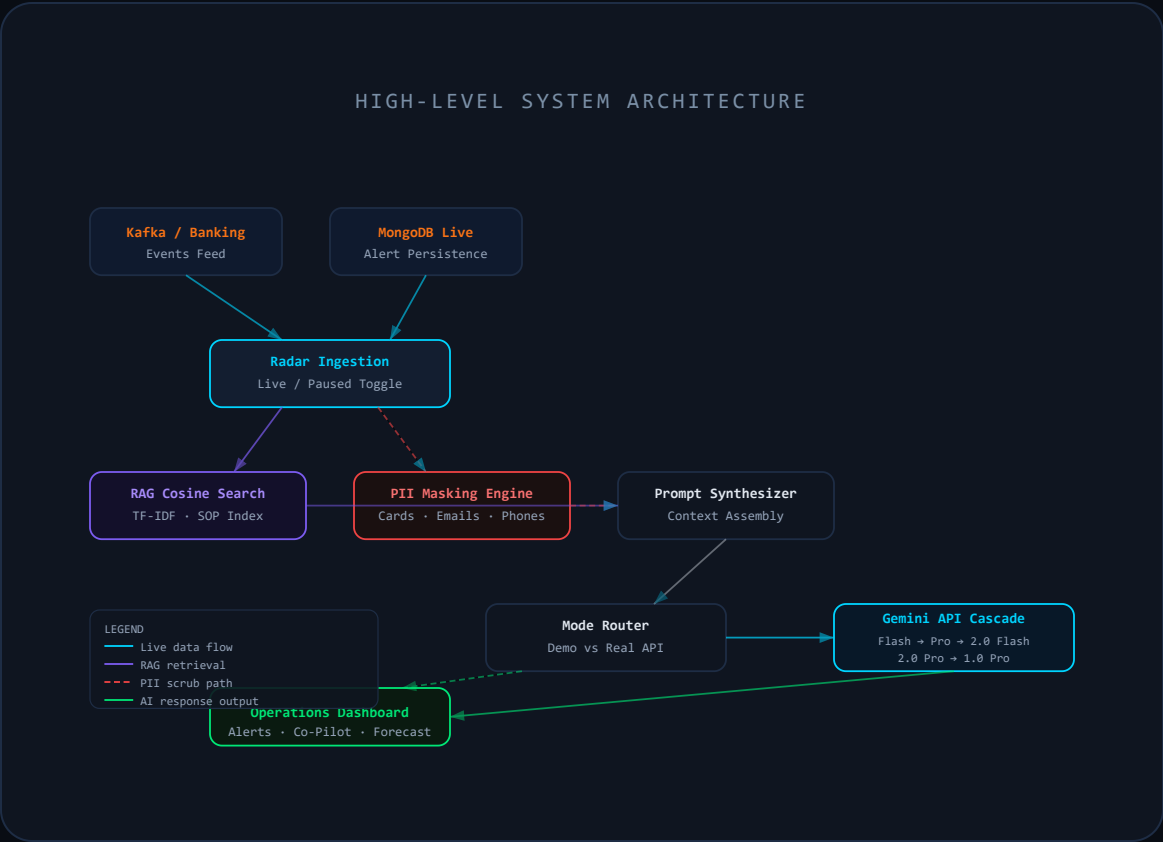


Model Rotation

Auto-cascades through 5 Gemini models on quota limits to maximize AI uptime with zero manual intervention.

System Architecture

FixFLOW AI is a full-stack application split into a React/Vite frontend and a Node.js/Express backend proxy. All AI calls are server-side — the browser never directly contacts the Gemini API.



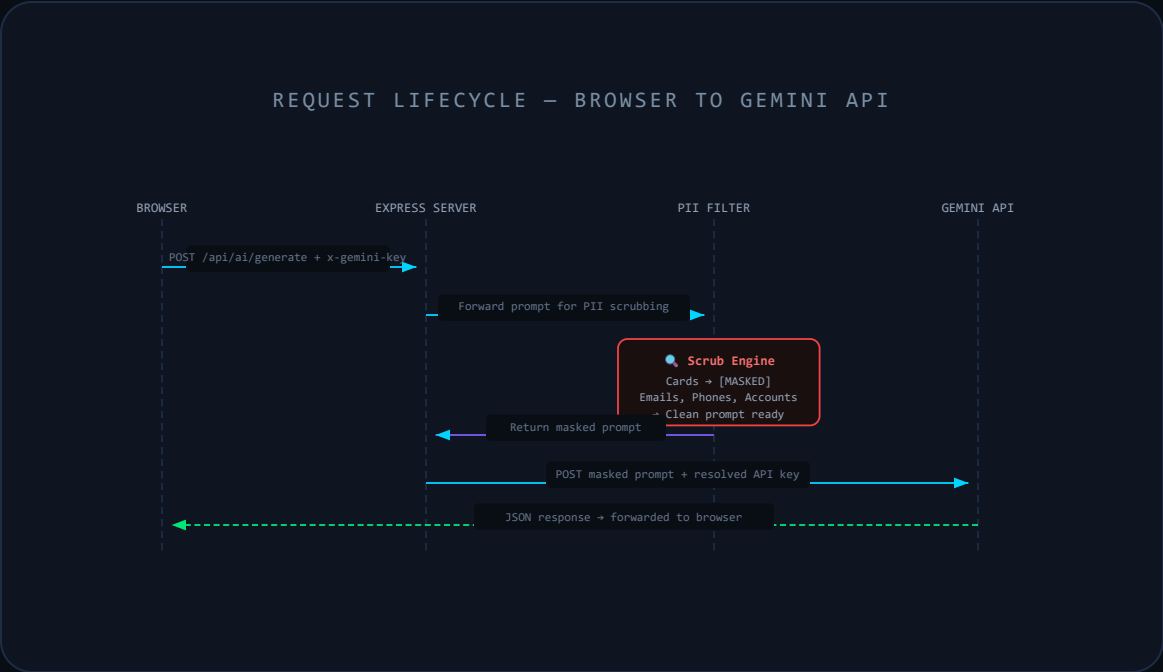
Technology Stack

LAYER	TECHNOLOGY	PURPOSE
Frontend	React 18 + Vite	SPA dashboard with real-time alert feed, chat interface, and forecast views
Backend	Node.js + Express 5	Secure API proxy, PII masking middleware, Gemini routing
Database	MongoDB (Mongoose)	Alert and incident persistence across sessions
AI Engine	Google Gemini API	Natural language incident analysis and operational forecasting

LAYER	TECHNOLOGY	PURPOSE
RAG Index	TF-IDF + Cosine Sim	Local SOP retrieval without external data exposure
Hosting	Render.com	Full-stack Node web service serving compiled Vite assets

03 · DATA FLOW

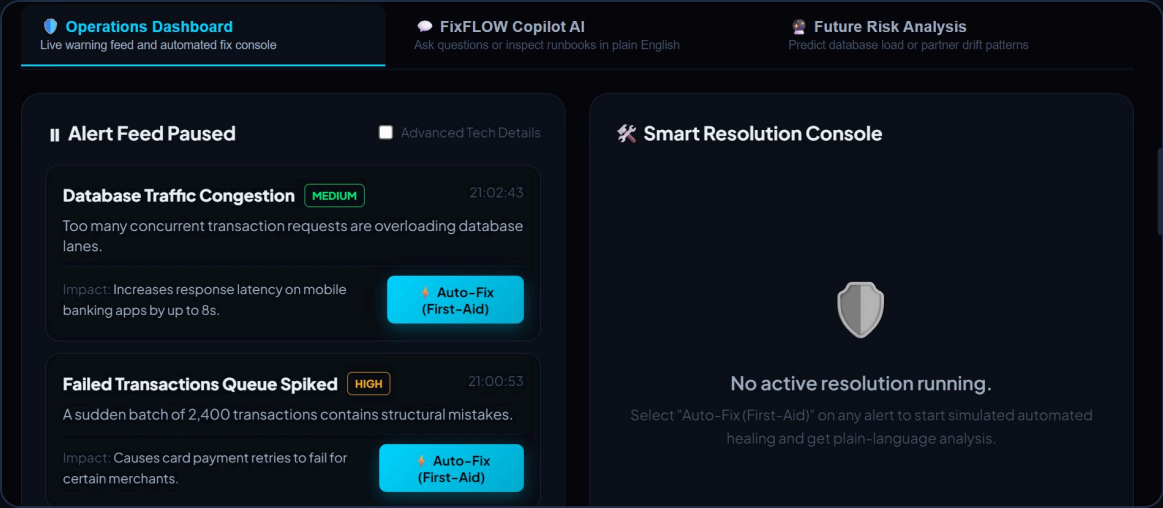
Data Flow & Pipeline



Radar Ingestion Pipeline

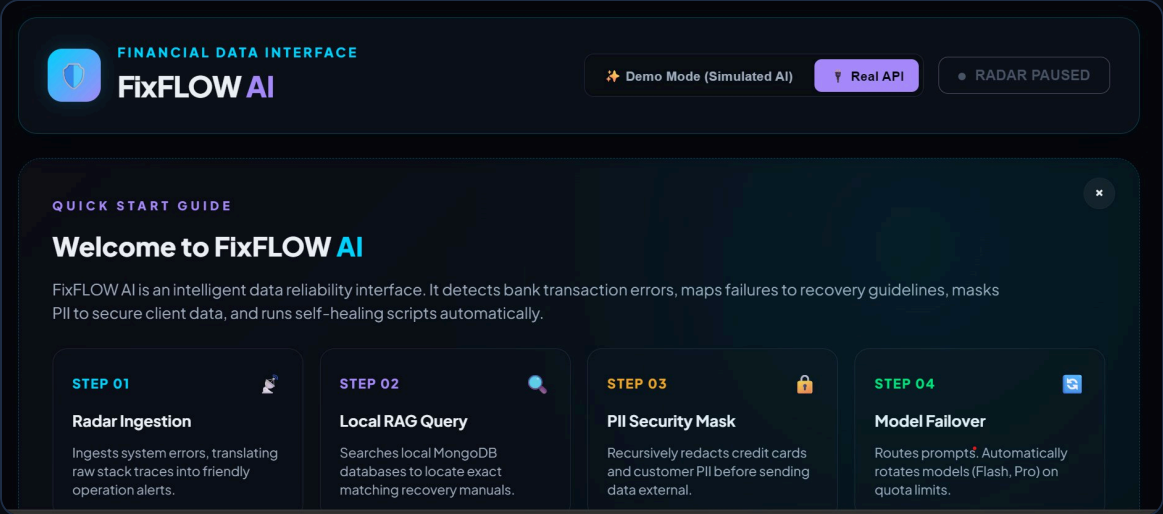
Banking events arrive as JSON/XML payloads and are ingested by the Live Radar module. The radar can be toggled live or paused by operators — when live, it polls MongoDB for new alerts and translates raw error codes into human-readable descriptions (e.g., `DEAD_LETTER` → "Failed Items Queue Spike").

FIG 1 — OPERATIONS DASHBOARD: LIVE ALERT FEED



Alert cards show severity badges (HIGH/MEDIUM), plain-language descriptions, and one-click Auto-Fix buttons. The Smart Resolution Console on the right runs interactive remediation scripts.

FIG 2 — RADAR TOGGLE: LIVE VS PAUSED MODE



Header controls allow operators to switch between Demo Mode (simulated AI) and Real API, and toggle the live radar on or off.

Security & PII Masking

Every prompt is intercepted server-side before reaching Google's infrastructure. The recursive masking engine traverses nested JSON bodies to scrub all sensitive tokens regardless of nesting depth.

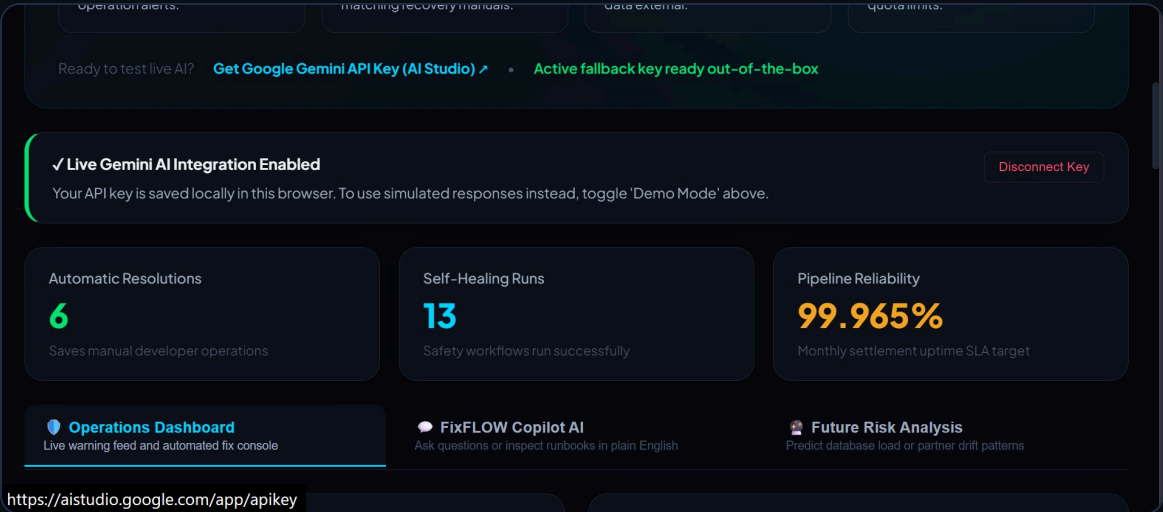
Masking Rules

DATA TYPE	REPLACEMENT TOKEN	PATTERN COVERAGE
Credit Cards	[MASKED_CREDIT_CARD]	13–16 digit sequences, space/dash separated
Bank Accounts	[MASKED_ACCOUNT_NUMBER]	9–18 digit account IDs
Email Addresses	[MASKED_EMAIL]	Standard RFC email pattern, 2–7 char TLD
Phone Numbers	[MASKED_PHONE_NUMBER]	International format, optional country code

API Key Security

The browser never directly contacts generativelanguage.googleapis.com. User-provided keys travel in the x-gemini-key header to the Express proxy — never in query strings or response bodies visible in browser DevTools.

FIG 3 — GEMINI API KEY CONFIGURATION



Users can supply their own Gemini API key (stored in `localStorage`) or rely on the server-side fallback key. KPI cards show live system metrics: 6 automatic resolutions, 13 self-healing runs, 99.965% pipeline reliability.

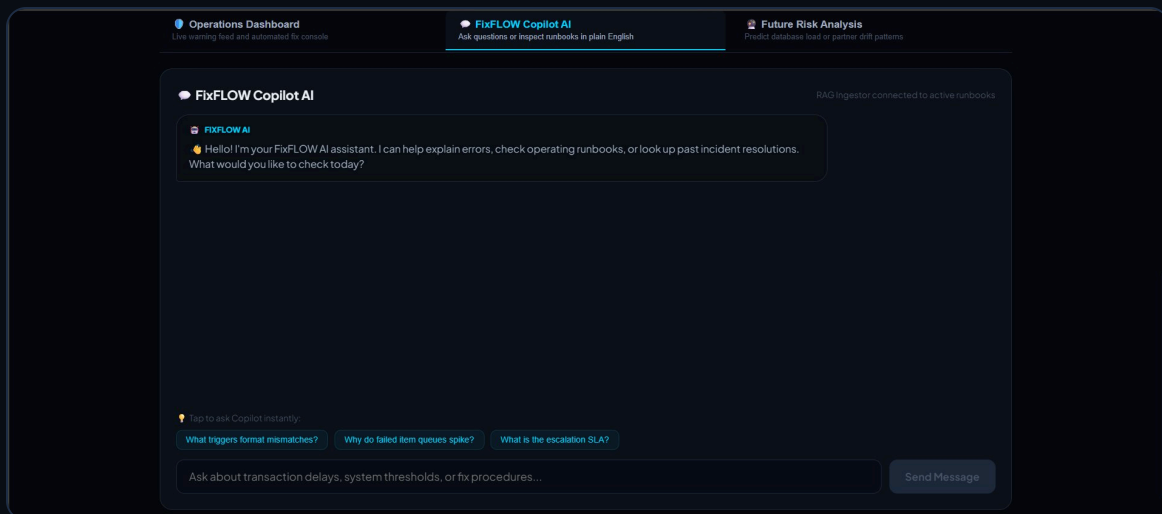
05 · FEATURES

Feature Walkthrough

FixFLOW Copilot AI — Chat Interface

The Copilot tab provides an interactive Q&A terminal backed by the RAG knowledge base and Gemini API. Operators can ask natural-language questions about transaction delays, SLA limits, system thresholds, or fix procedures. Pre-loaded quick-prompts accelerate common queries.

FIG 4 — FIXFLOW COPILOT AI CHAT

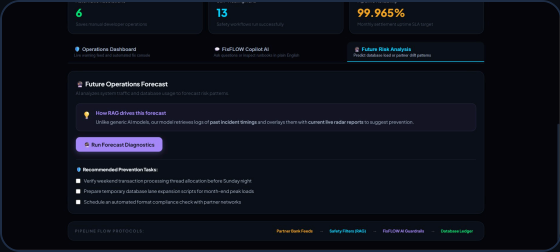


The copilot greets operators and surfaces quick-tap prompts ("What triggers format mismatches?", "Why do failed item queues spike?"). The RAG Ingestor is connected to active runbooks in the top-right status.

Future Risk Analysis — Forecast Tab

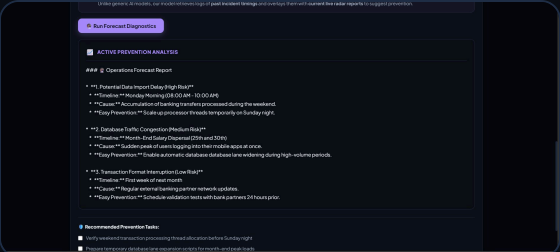
The forecast engine retrieves logs of past incident timings from MongoDB and overlays them with current live radar data to predict upcoming failure windows. Running "Forecast Diagnostics" generates a structured Operations Forecast Report with risk levels and recommended prevention tasks.

FIG 5 — FUTURE OPERATIONS FORECAST PANEL



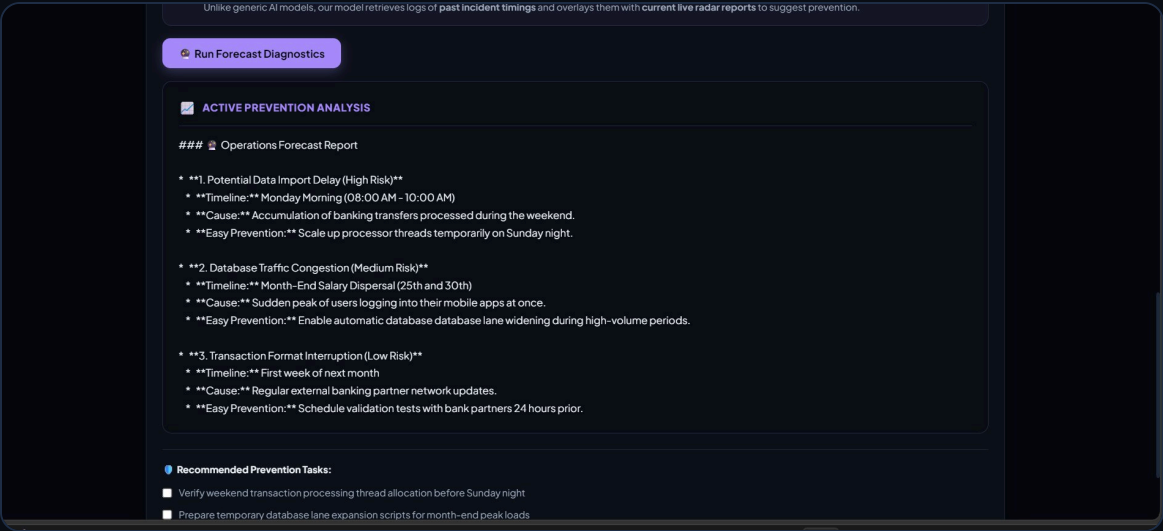
Forecast view showing RAG-driven prediction panel and recommended prevention task checklist.

FIG 6 — ACTIVE PREVENTION ANALYSIS REPORT



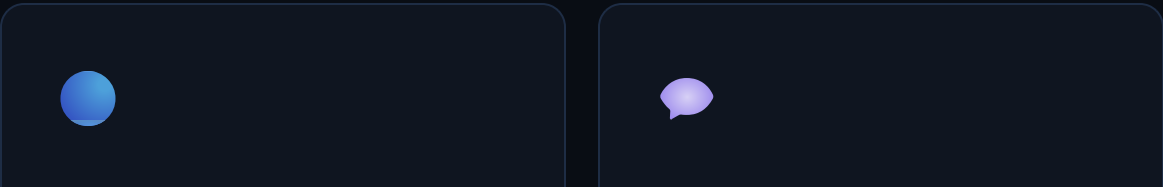
Expanded forecast report with three risk categories: High (Data Import Delay), Medium (DB Traffic Congestion), and Low (Transaction Format Interruption).

FIG 7 — DETAILED ACTIVE PREVENTION ANALYSIS



Each risk entry includes predicted timeline, root cause, and an easy prevention step — e.g., "Scale up processor threads temporarily on Sunday night" for the Monday morning banking transfer accumulation risk.

Three-Tab Navigation



Operations Dashboard

Live alert feed, severity classification, Auto-Fix (First-Aid) console, and Smart Resolution runner.

FixFLOW Copilot AI

RAG-augmented AI chat for runbook lookups, SLA queries, and plain-language error explanations.



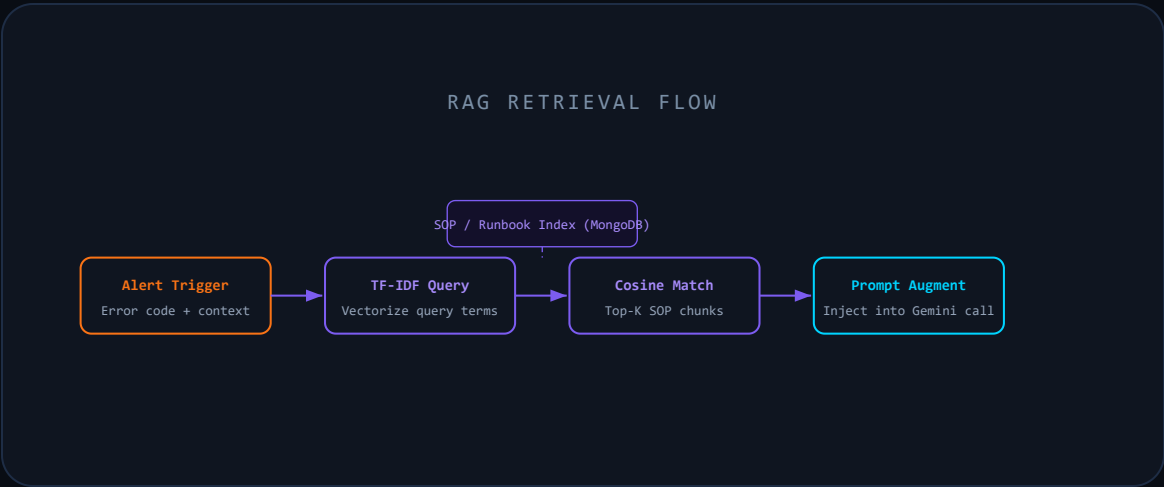
Future Risk Analysis

Predictive diagnostics that merge historical incident data with live radar patterns to forecast upcoming risks.

06 · RAG SYSTEM

RAG Knowledge Base

FixFLOW AI uses a locally-indexed Retrieval-Augmented Generation pipeline. SOP manuals and runbooks are stored in MongoDB and vectorized using TF-IDF weighting with cosine similarity scoring. When an alert fires, the top matching SOP chunks are retrieved and injected into the Gemini prompt — no raw SOP text ever leaves the server unfiltered.



Reference Library (Open Source Tools)

CATEGORY	TOOL	PURPOSE
SSL/TLS	Let's Encrypt	Free automated certificate authority for web servers
Intrusion Prevention	Fail2ban	Log monitoring and IP banning for malicious activity
PII Detection	Microsoft Presidio	Customizable PII anonymisation for logs and AI prompts
Test Data	Faker.js	High-volume realistic mock datasets without real PII exposure
Proxy / Gateway	Nginx	Reverse proxy with built-in rate limiting and connection throttling
DB Pooling	PgBouncer	Connection pool guard against PostgreSQL exhaustion spikes
DDoS / WAF	Cloudflare Free	DDoS prevention, SSL termination, and WAF rule sets

07 · SETUP

Setup & Deployment

Local Development

```
npm install

# Create .env in project root:
```

```
PORT=5000
MONGODB_URI=mongodb+srv://agent:agent1234@cluster0.jgqk0om.mongodb.net/fixflow
GEMINI_API_KEY=your_gemini_api_key_here
VITE_GEMINI_API_KEY=your_gemini_api_key_here

npm run dev      # Vite frontend + Node backend (concurrent)
npm run start    # Production server
```

Deploy to Render



1

Create Web Service

Link your GitHub repository to a new Render Web Service; select Node as the runtime.

2

Build & Start Commands

Build: `npm install && npm run build` | Start: `npm run start`

3

Environment Variables

Add `MONGODB_URI` and `VITE_GEMINI_API_KEY` in the Render Environment tab.

4

Deploy

Render automatically builds and serves the compiled Vite static assets via Express.

Model Rotation Cascade

On any `429 Quota Exceeded` response, the backend automatically tries the next model in sequence:

```
1. gemini-1.5-flash      ← default
2. gemini-1.5-pro
3. gemini-2.0-flash
4. gemini-2.0-pro
5. gemini-1.0-pro      ← final fallback
```

Module Reference

Frontend (src/App.jsx)

COMPONENT / FEATURE	DESCRIPTION
Operations Dashboard	Live radar feed, alert cards with severity, Auto-Fix console, Smart Resolution terminal
FixFLOW Copilot AI	RAG-backed chat terminal; quick-prompt chips; plain-English runbook lookup
Future Risk Analysis	Predictive forecast with RAG overlay on historical incident logs; prevention task checklist
Onboarding Banner	4-step Quick Start Guide (closable, retriggerable via header button)
Credential Config	Gemini key input caching in localStorage; sends via <code>x-gemini-key</code> header
Quota Alert Box	Glowing red warning on 429 errors with option to switch to Demo Mode

Backend (server.js)

MODULE	DESCRIPTION
PII Masking Middleware	Recursive JSON body scanner; redacts cards, accounts, emails, phones before any external call
POST /api/ai/generate	Main proxy route: scrubs → resolves API key → routes to Gemini with model cascade fallback
Mongoose Schema	Alert and incident models persisted in MongoDB across sessions

MODULE	DESCRIPTION
<code>Vite</code> <code>Catch-all</code> <code>Route</code>	RegExp literal <code>/.*/</code> routing for Express 5 compatibility with compiled SPA assets
FixFLOW AI · Technical Documentation v1.0	fixflow-ai-z7zj.onrender.com ↗ Built with React · Node.js · MongoDB · Gemini API